vae_notebook

April 19, 2026

We would like to acknowledge University of Michigan's EECS 498-007/598-005 on which we based the development of this project.

1 Variational Autoencoder

In this notebook, you will implement a variational autoencoder (VAE) and a conditional variational autoencoder (CVAE) and apply them to the MNIST handwritten digit dataset. An autoencoder learns a compact latent representation of training images and reconstructs them from that representation. The VAE extends this by making the encoder and decoder probabilistic, allowing us to sample from the learned latent distribution to generate new images.

1.1 Assignment Overview

All your code goes in `vae.py`. The notebook imports from it and runs checks — do not modify the notebook cells.

Task	Points
`VAE.__init__` — encoder, `mu_layer`, `logvar_layer`	4
`VAE.__init__` — decoder	1
`VAE.forward`	1
`reparametrize`	2
`loss_function`	1
`CVAE.__init__` + `CVAE.forward`	3
Total	**12**

`train_vae`, `show_images`, `reset_seed`, `rel_error`, and `one_hot` are all provided in `vae_utils.py` — do not modify that file.

[1]:
```
import os
import sys
sys.path.insert(0, os.path.dirname(os.path.abspath('vae.py')))
print('cwd:', os.getcwd())
```

```
cwd: /Users/kendallkikkawa/Documents/UCLA/ECE_247B/ucla-engr-c147b-2026-spring-project-1/VAE
```

[2]:
```
%load_ext autoreload
%autoreload 2
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision.transforms as T
import torchvision.datasets as dset
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

from vae_utils import rel_error, reset_seed, show_images, train_vae,␣
↪count_params

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0)
plt.rcParams['font.size'] = 16
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

device = (
    torch.device('cuda') if torch.cuda.is_available()
    else torch.device('mps') if torch.backends.mps.is_available()
    else torch.device('cpu')
)
print('Using device:', device)
```

```
Using device: cpu
```

1.2 Load MNIST Dataset

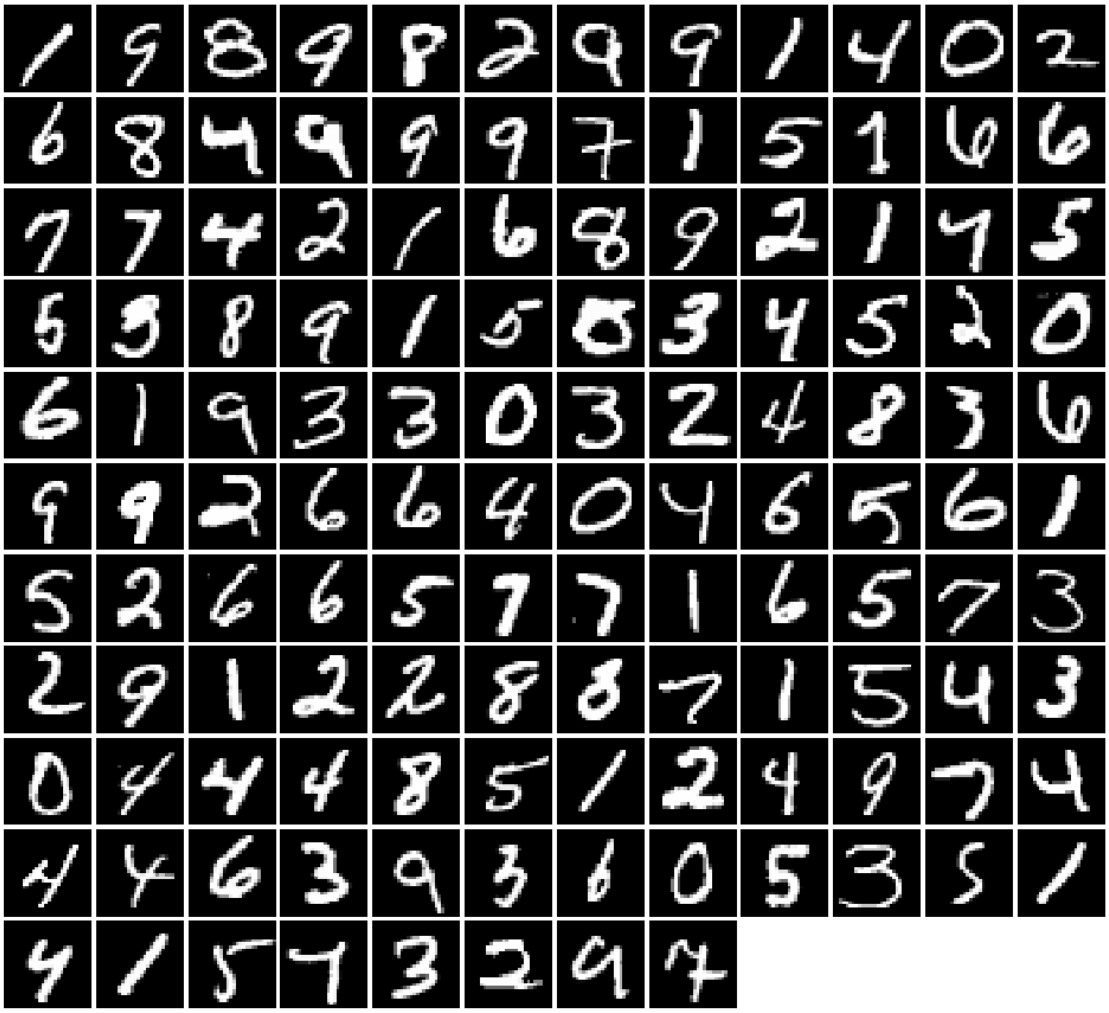
VAEs are notoriously finicky with hyperparameters, and also require many training epochs. In order to make this assignment approachable, we will be working on the MNIST dataset, which is 60,000 training and 10,000 test images. Each picture contains a centered image of white digit on black background (0 through 9). The data will be saved into a folder called `MNIST` on first run.

[3]:
```
batch_size = 128

mnist_train = dset.MNIST('./mnist_data', train=True, download=True, transform=T.
↪ToTensor())
loader_train = DataLoader(mnist_train, batch_size=batch_size, shuffle=True,␣
↪drop_last=True, num_workers=0)
```

1.3 Visualize Dataset

[4]:
```
imgs = next(iter(loader_train))[0]
show_images(imgs)
```



2 Fully Connected VAE

Our first VAE implementation will consist solely of fully connected layers. We'll take the `1 x 28 x 28` shape of our input and flatten the features to create an input dimension size of 784. In this section you'll define the encoder and decoder in the `VAE` class in `vae.py`, implement the reparametrization trick, the forward pass, and the loss function.

2.1 FC-VAE Encoder (4 points)

Build the encoder inside `VAE.__init__` in `vae.py`. Set `self.hidden_dim = 400` and define: - `self.encoder` — an `nn.Sequential` that maps `(N, 1, H, W)` → `(N, hidden_dim)`

- `self.mu_layer` — a single `nn.Linear` that maps `(N, hidden_dim)` → `(N, Z)` - `self.logvar_layer` — a single `nn.Linear` that maps `(N, hidden_dim)` → `(N, Z)`

Encoder architecture: * `nn.Flatten()` * Linear(`input_size` → `hidden_dim`) + ReLU * Linear(`hidden_dim` → `hidden_dim`) + ReLU * Linear(`hidden_dim` → `hidden_dim`) + ReLU

[5]:
```
from vae import VAE

def test_encoder(model, input_size, hidden_dim, n_encoder_lin_layers):
    expected = (input_size + 1) * hidden_dim + (n_encoder_lin_layers - 1) *␣
 ↪(hidden_dim + 1) * hidden_dim
    actual = count_params(model.encoder)
    if actual == expected:
        print('Correct number of parameters in model.encoder.')
    else:
        print('Incorrect number of parameters in model.encoder. (mu_layer and␣
 ↪logvar_layer should be separate)')

def test_mu_logvar(model, hidden_dim, latent_size):
    for name, layer in [('mu_layer', model.mu_layer), ('logvar_layer', model.
 ↪logvar_layer)]:
        expected = (hidden_dim + 1) * latent_size
        actual = count_params(layer)
        status = 'Correct' if actual == expected else 'Incorrect'
        print(f'{status} number of parameters in model.{name}.')

test_encoder(VAE(345, 17), 345, 400, 3)
test_mu_logvar(VAE(345, 17), 400, 17)
```

```
Correct number of parameters in model.encoder.
Correct number of parameters in model.mu_layer.
Correct number of parameters in model.logvar_layer.
```

2.2 FC-VAE Decoder (1 point)

Build the decoder inside `VAE.__init__`. Define `self.decoder` as an `nn.Sequential` that maps `(N, Z)` → `(N, 1, H, W)`.

Decoder architecture: * Linear(`latent_size` → `hidden_dim`) + ReLU * Linear(`hidden_dim` → `hidden_dim`) + ReLU * Linear(`hidden_dim` → `hidden_dim`) + ReLU * Linear(`hidden_dim` → `input_size`) + Sigmoid * `nn.Unflatten(1, (1, 28, 28))`

[6]:
```
from vae import VAE

def test_decoder(model, input_size, hidden_dim, latent_size,␣
 ↪n_decoder_lin_layers):
    expected = (
        (latent_size + 1) * hidden_dim
        + (n_decoder_lin_layers - 2) * (hidden_dim + 1) * hidden_dim
```

```
        + (hidden_dim + 1) * input_size
    )
    actual = count_params(model.decoder)
    status = 'Correct' if actual == expected else 'Incorrect'
    print(f'{status} number of parameters in model.decoder.')

test_decoder(VAE(345, 17), 345, 400, 17, 4)
```

```
Correct number of parameters in model.decoder.
```

2.3 Reparametrization (2 points)

To sample a latent vector z from $q_\phi(z|x) = \mathcal{N}(\mu, \sigma^2)$ in a differentiable way, we use the reparametrization trick:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1)$$

This lets gradients flow back through z to μ and σ during backpropagation.

Implement `reparametrize` in `vae.py`. Verify that your mean and std errors are below `1e-4`.

[7]:
```
reset_seed(0)
from vae import reparametrize

latent_size = 15
size = (1, latent_size)
mu = torch.zeros(size)
logvar = torch.ones(size)

z = reparametrize(mu, logvar)

expected_mean = torch.FloatTensor([-0.4363])
expected_std = torch.FloatTensor([1.6860])
assert z.size() == size
print('Mean Error', rel_error(torch.mean(z, dim=-1), expected_mean))
print('Std Error',  rel_error(torch.std(z, dim=-1),  expected_std))
```

```
Mean Error 5.639056398351415e-05
Std Error 7.1412955526273885e-06
```

2.4 FC-VAE Forward (1 point)

Implement `VAE.forward` in `vae.py`. Pass the input through the encoder to get `mu` and `logvar`, reparametrize to get `z`, then decode `z` to reconstruct the image.

[8]:
```
from vae import VAE

def test_VAE_shapes():
    all_correct = True
```

```
    with torch.no_grad():
        batch_size = 3
        latent_size = 17
        x_hat, mu, logvar = VAE(28 * 28, latent_size)(torch.ones(batch_size, 1,␣
↪28, 28))
        for name, tensor, expected in [
            ('x_hat',  x_hat,  (batch_size, 1, 28, 28)),
            ('mu',     mu,     (batch_size, latent_size)),
            ('logvar', logvar, (batch_size, latent_size)),
        ]:
            if tuple(tensor.shape) != expected:
                print(f'{name}: expected {expected}, got {tuple(tensor.shape)}')
                all_correct = False
        if all_correct:
            print('Shapes of x_hat, mu, and logvar are correct.')
        if batch_size > 1 and x_hat.std(0).mean() == 0:
            print('Warning: x_hat has no randomness - check reparametrize.')

test_VAE_shapes()
```

```
Shapes of x_hat, mu, and logvar are correct.
```

2.5 Loss Function (1 point)

The VAE loss is the negative variational lower bound:

$$\mathcal{L} = \underbrace{-\mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)]}_{\text{reconstruction}} + \underbrace{D_{KL}(q_\phi(z|x)\|p(z))}_{\text{KL divergence}}$$

The KL term has a closed form for diagonal Gaussians vs. a standard normal prior:

$$D_{KL} = -\frac{1}{2}\sum_{j=1}^{Z}\left(1 + \log\sigma_j^2 - \mu_j^2 - \sigma_j^2\right)$$

Implement `loss_function` in `vae.py`. Use `F.binary_cross_entropy(x_hat, x, reduction='sum')` for the reconstruction term. Average the total loss over the batch. Your relative error should be `1e-5`.

[9]:
```
from vae import loss_function

image_hat = torch.sigmoid(torch.FloatTensor([[2, 5], [6, 7]]).unsqueeze(0).
↪unsqueeze(0))
image     = torch.sigmoid(torch.FloatTensor([[1, 10], [9, 3]]).unsqueeze(0).
↪unsqueeze(0))
image_hat = torch.tile(image_hat, (10, 1, 1, 1))
image     = torch.tile(image, (10, 1, 1, 1))
```

```
mu_shape = (image.shape[0], 15)
mu, logvar = torch.ones(mu_shape), torch.zeros(mu_shape)

expected_out = torch.tensor(1.0079 + 7.5000)
out = loss_function(image_hat, image, mu, logvar)
print('Loss error', rel_error(expected_out, out))
```

```
Loss error 2.1297676389877955e-06
```

2.6 Train VAE

`train_vae` is provided in `vae_utils.py`. Training for 10 epochs should take ~2 minutes and your loss should be less than 120.

[10]:
```
from vae import VAE

num_epochs = 10
latent_size = 15
input_size = 28 * 28

vae_model = VAE(input_size, latent_size=latent_size).to(device)
optimizer = optim.Adam(vae_model.parameters(), lr=1e-3)

for epoch in range(num_epochs):
    train_vae(epoch, vae_model, loader_train, optimizer)
```

```
Epoch 0: loss = 191.1649
Epoch 1: loss = 151.7652
Epoch 2: loss = 135.6218
Epoch 3: loss = 128.0424
Epoch 4: loss = 124.7236
Epoch 5: loss = 122.5418
Epoch 6: loss = 120.8761
Epoch 7: loss = 119.0738
Epoch 8: loss = 117.1429
Epoch 9: loss = 115.4299
```

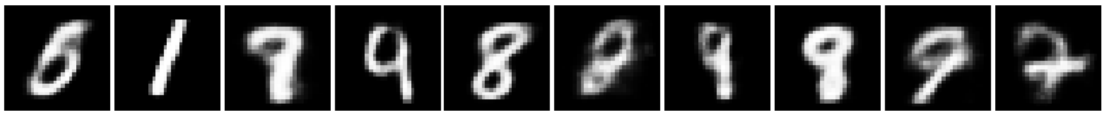
2.7 Visualize Results

Sample random latent vectors and decode them to generate new images. You should be able to visually recognize many digits, though some may be blurry.

[11]:
```
vae_model.eval()
with torch.no_grad():
    z = torch.randn(10, latent_size, device=device)
    samples = vae_model.decoder(z).cpu()

fig = plt.figure(figsize=(10, 1))
gspec = gridspec.GridSpec(1, 10)
```

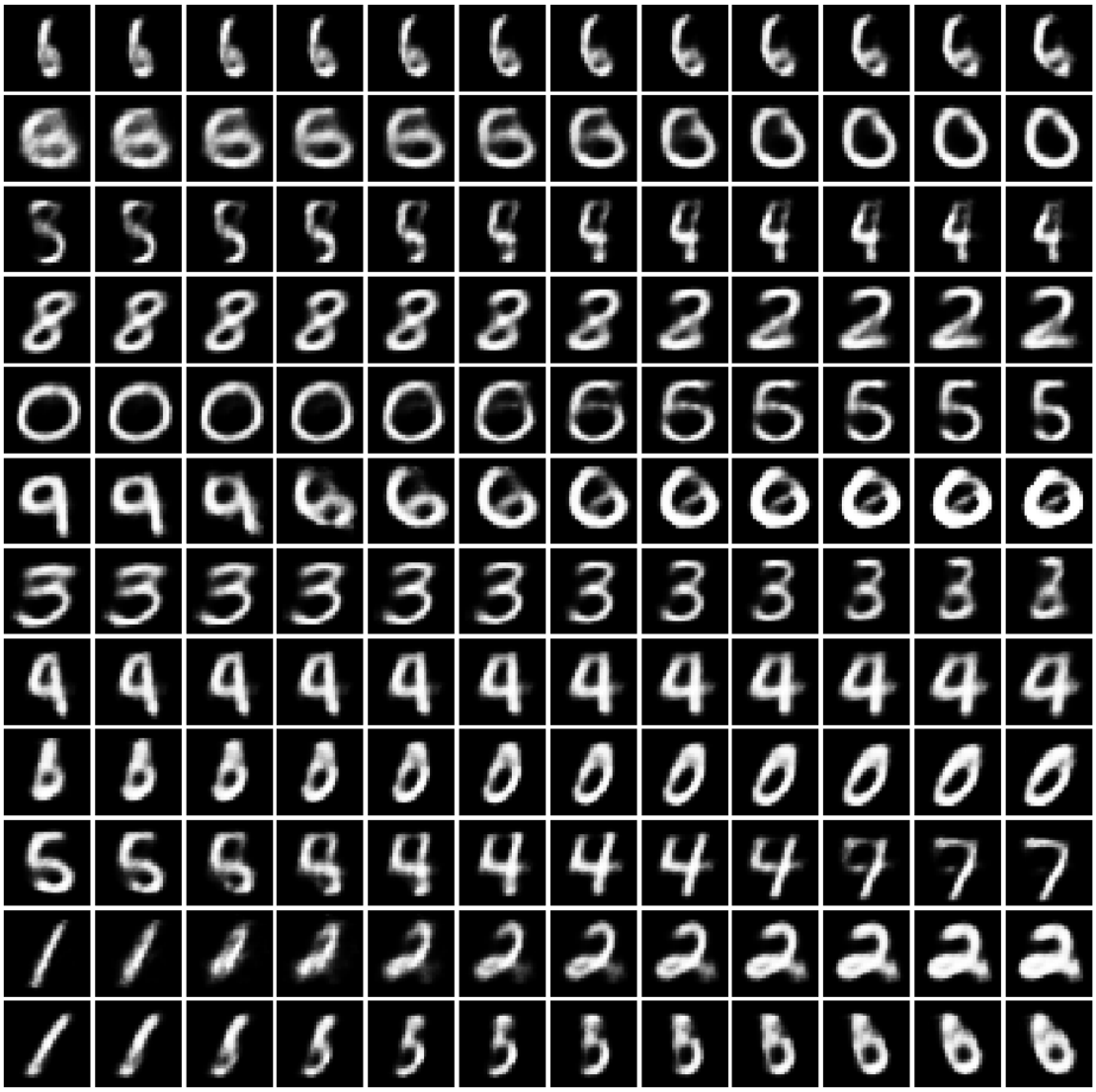
```
gspec.update(wspace=0.05, hspace=0.05)
for i, sample in enumerate(samples):
    ax = plt.subplot(gspec[i])
    plt.axis('off')
    ax.set_aspect('equal')
    plt.imshow(sample.reshape(28, 28), cmap='Greys_r')
```



2.8 Latent Space Interpolation

Generate random latent vectors z_0 and z_1 and linearly interpolate between them. Each row below interpolates between two random vectors — you should see smooth transitions between digits.

[12]:
```
S = 12
vae_model.eval()
with torch.no_grad():
    z0 = torch.randn(S, latent_size, device=device)
    z1 = torch.randn(S, latent_size, device=device)
    w  = torch.linspace(0, 1, S, device=device).view(S, 1, 1)
    z  = (w * z0 + (1 - w) * z1).transpose(0, 1).reshape(S * S, latent_size)
    x  = vae_model.decoder(z).cpu()
show_images(x)
```



3 Conditional FC-VAE

The CVAE extends the VAE by conditioning on the class label c. Instead of $q_\phi(z|x)$ and $p_\theta(x|z)$ we have $q_\phi(z|x,c)$ and $p_\theta(x|z,c)$. This lets us generate specific digits at inference time by choosing c.

3.1 CVAE (3 points)

Implement `CVAE.__init__` and `CVAE.forward` in `vae.py`. Use the same architecture as the VAE with these modifications:

1. **Encoder** input: concatenation of flattened image `(N, H*W)` and one-hot label `(N, K)` → first linear layer takes `H*W + K`. Do **not** put `nn.Flatten` inside the encoder Sequential — flatten in `forward` instead.

2. **Decoder** input: concatenation of latent vector `z (N, Z)` and one-hot label `(N, K)` → first linear layer takes `Z + K`.
3. **Forward**: flatten `x`, `torch.cat` with `labels` before encoder; `torch.cat z` with `labels` before decoder.

[20]:
```
from vae import CVAE

def test_CVAE_shapes():
    all_correct = True
    with torch.no_grad():
        batch_size = 3
        num_classes = 10
        latent_size = 17
        cls = F.one_hot(torch.tensor([3] * batch_size),␣
↪num_classes=num_classes).float()
        x_hat, mu, logvar = CVAE(28 * 28, num_classes=num_classes,␣
↪latent_size=latent_size)(
            torch.ones(batch_size, 1, 28, 28), cls
        )
        for name, tensor, expected in [
            ('x_hat',  x_hat,  (batch_size, 1, 28, 28)),
            ('mu',     mu,     (batch_size, latent_size)),
            ('logvar', logvar, (batch_size, latent_size)),
        ]:
            if tuple(tensor.shape) != expected:
                print(f'{name}: expected {expected}, got {tuple(tensor.shape)}')
                all_correct = False
        if all_correct:
            print('Shapes of x_hat, mu, and logvar are correct.')
        if batch_size > 1 and x_hat.std(0).mean() == 0:
            print('Warning: x_hat has no randomness.')

test_CVAE_shapes()
```

```
Shapes of x_hat, mu, and logvar are correct.
```

3.2 Train CVAE

Training for 10 epochs should take ~2 minutes and your loss should be less than 120.

[21]:
```
from vae import CVAE

num_epochs = 10
latent_size = 15
input_size = 28 * 28

cvae = CVAE(input_size, latent_size=latent_size).to(device)
optimizer = optim.Adam(cvae.parameters(), lr=1e-3)
```

```
for epoch in range(num_epochs):
    train_vae(epoch, cvae, loader_train, optimizer, cond=True)
```

```
Epoch 0: loss = 177.1467
Epoch 1: loss = 135.0875
Epoch 2: loss = 126.1131
Epoch 3: loss = 121.9592
Epoch 4: loss = 119.2012
Epoch 5: loss = 115.6657
Epoch 6: loss = 112.9438
Epoch 7: loss = 111.4149
Epoch 8: loss = 110.4176
Epoch 9: loss = 109.4971
```

3.3 Visualize CVAE Results

We can now conditionally generate one sample per digit (0–9) by providing the corresponding one-hot label. Each digit should be reasonably recognizable.

[22]:
```
cvae.eval()
with torch.no_grad():
    z = torch.randn(10, latent_size)
    c = torch.eye(10, 10)  # one-hot labels for digits 0-9
    zc = torch.cat((z, c), dim=-1).to(device)
    samples = cvae.decoder(zc).cpu()

fig = plt.figure(figsize=(10, 1))
gspec = gridspec.GridSpec(1, 10)
gspec.update(wspace=0.05, hspace=0.05)
for i, sample in enumerate(samples):
    ax = plt.subplot(gspec[i])
    plt.axis('off')
    ax.set_aspect('equal')
    plt.imshow(sample.reshape(28, 28), cmap='Greys_r')
```

[ ]:
```

```